

Overview

🎯 In this course, you will explore how to import and view test results in Squish Test Center using a variety of methods from uploading via command line to manual testing.

What Will I Learn ?

1. Understand the fundamental elements of how to get started with Squish Test Center.
2. Understand and explore the different views in the Squish Test Center UI
3. How to upload test results from the Squish IDE.
4. How to upload test results from the Squish Test Center UI.
5. How to upload test results from the command line.
6. How to manually conduct tests in Squish Test Center.

[GitHub - qt-learning/Test-Results · GitHub](#)

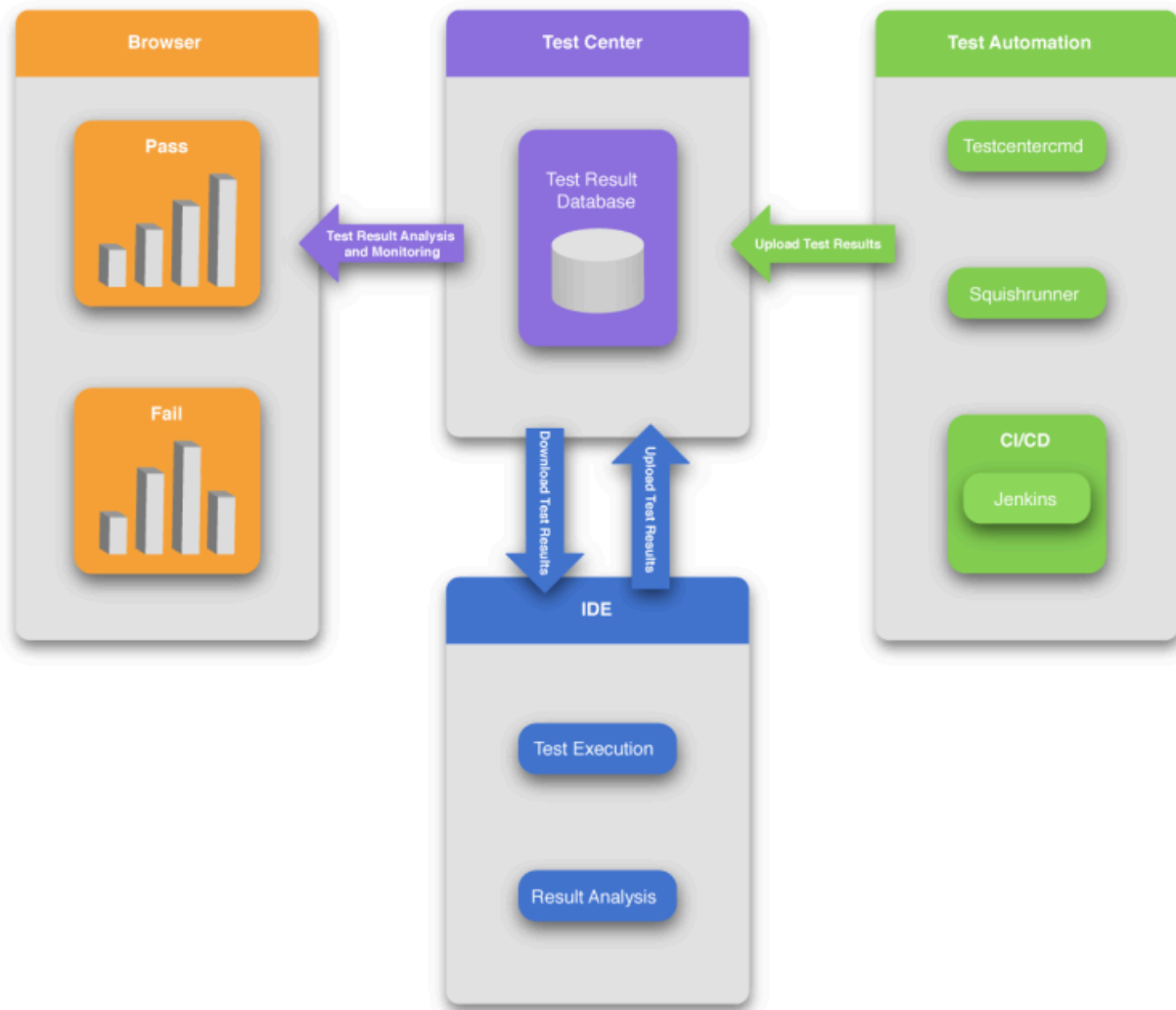
[Views | Test Center Manual](#)

[Manage users | Test Center Manual](#)

[3rd-party integration | Test Center Manual](#)

Getting Started with Squish Test Center

Squish Test Center is a centralized repository to store test results. This repository is a results server which allows users to browse the stored test results and statistics via web browser. The Squish Test Center application binary is included in the Squish installation folder. Additionally, it can be downloaded from a [Qt Account](#) for those using any 3rd party testing tools.



The Squish IDE is integrated with Squish Test Center, so you can upload tests executed from Squish IDE to Squish Test Center. You can also download and view results from Squish Test Center within Squish IDE.

Since Squish Test Center offers many different ways to upload results, it's easy to integrate it into many different work flows, from single user testing on a local machine to hundreds of tests executed in parallel by a continuous integration system.

STEPS - Setting Up Squish Test Center

STEP 01 = Configuring the host settings

Shell

```
## ActionMailer SMTP section
##

# Specify the connection's host name or IP address.
ActionMailer.smtp.HostName=smtp.YOURHOSTNAME.com

# Specify the connection's port number.
ActionMailer.smtp.Port=25
```

In the **application.ini** file, the **ActionMailer.smtp.HostName** value was specified as the connection's host name or IP address, and the **ActionMailer.smtp.Port** value was specified as the connection's port number. 'YOURHOSTNAME' refers to the name of your computer.

STEP 02 = Starting Test Center

None

```
C:\squish-testcenter-3.3.0-windows-x64\bin>testcenter start
Reading database configuration for environment "product" from
"C:/squish-testcenter-3.3.0-windows-x64/testcenter/config/database.ini"
```

```
DriverType: "SQLITE"  
DatabaseName: "db/teamserver.dbfile"  
Absolute SQLite file path "C:/squish-testcenter-3.3.0-windows-x64  
/testcenter/db/teamserver.dbfile"  
Connect Options: "SQLITE_BUSY_TIMEOUT=2147483647"  
SQLite version: 3.24.0  
Current Schema Version: "202306151745"  
Latest Schema Version: "202306151745"  
Test Center started, listening on port 8800
```

We started the Test Center from the command line by issuing the command '**testcenter start**'. The Test Center server connection was confirmed as successful as we saw the output "**Test Center started, listening on port 8800**". Test Center is accessed via web browser with a URL that matches the host name and port number of the server. In our case, it was accessed by navigating to **localhost:8800**.

STEP 03 = Obtaining the activation code

Create the first account

LOGIN

Email: *

Full Name:

PASSWORD

ROLES

Choose roles: *

☒ Admin

☒ User

☒ Upload

Reset

 or

Add User

After activating Test Center, a dialog appeared to set up a user account for Test Center and set their permissions. A Name, Email, and Password had to be chosen in this dialog, which served as the login for this user. Some additional permission settings could be selected for this user as well by checking the “Choose rights” check boxes. We set up an account as an Admin user.

Uploading Test Results Using the Squish IDE

There are several ways to upload test results to Squish Test Center. Test Center supports different file formats and techniques for uploading the test results. There are three main ways to upload test results to Squish Test Center:

- Uploading from the Squish IDE
- Uploading manually from Squish Test Center UI
- Uploading from the command line using “testecentercmd”

In this section, we will focus on uploading test results from the Squish IDE.

Before discussing the process of uploading test results, it is important to understand the test results and the file formats supported by Test Center.



Squish Test Center groups test results based on batches, reports, and labels. A batch is a group of one or more reports, encompassing one or more test suites and configurations.

Squish users tend to organize their test cases into test suites, with each suite targeting a key feature of their project. A collection of one or more test results is viewed by Squish Test Center as a report.

If a single test result file is obtained after a test run, such as executing a single test suite on a single machine, the result file should be uploaded as a distinct batch. Conversely, if multiple test result files are obtained after a test run, such as executing multiple test suites or running tests on multiple machines and configurations, all result files pertaining to the same test run should be uploaded within a unified batch.

Reports can be tagged with and grouped by a **label**. For example, if the test executions within a report share common factors like product version, operating system and compiler.

In case there are additional characteristics that set apart the setups on which the tests are performed, such as the version of the application under test or the browser version, a Label can be assigned for each of them.

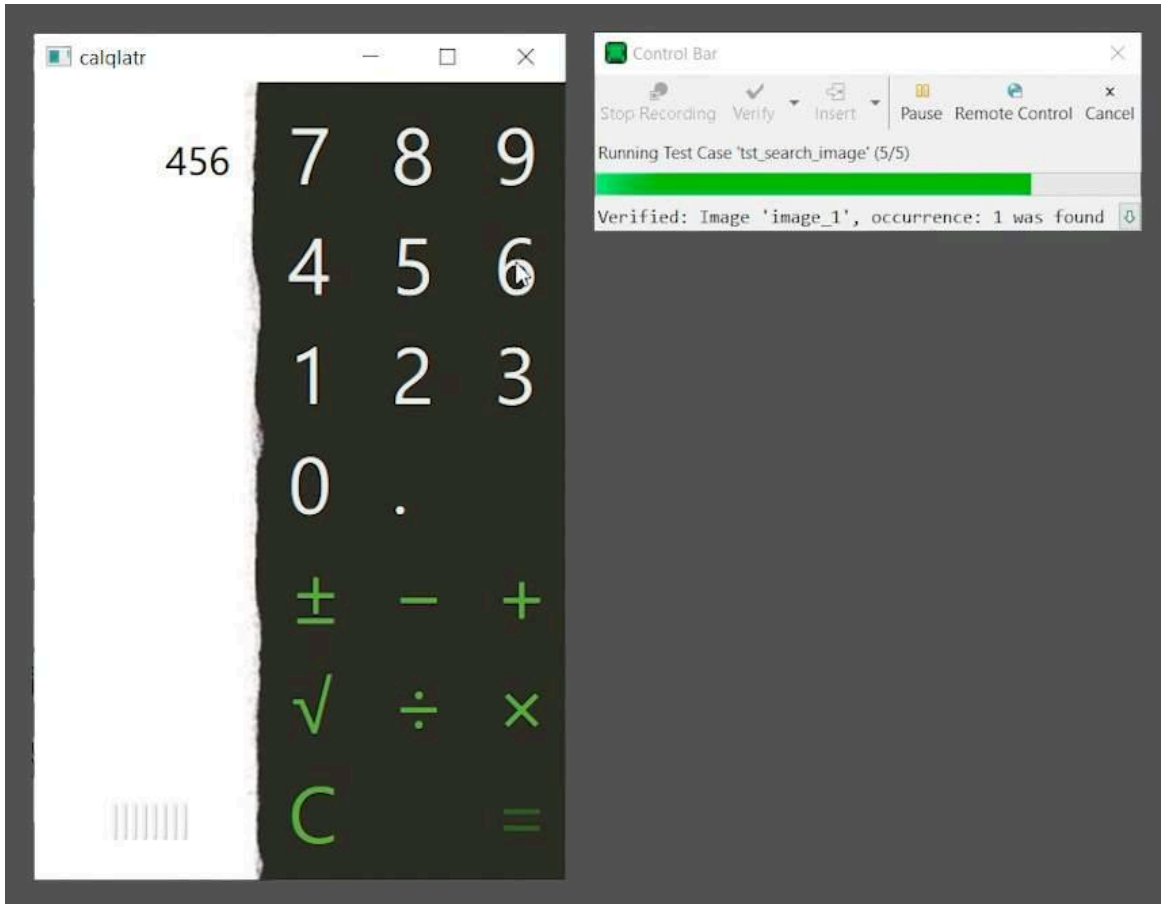
Once you have a batch of test results, they can be exported in the following formats:

Squish XML	XML is the default test report file format generated by the Squish IDE which can be uploaded to Test Center. This XML file contains information about the tests passes and failures. Additional test information such as screenshots, snapshots, or attachments are contained within a zip archive. generated alongside the XML file. It is recommended to use XML report version 3.4, but older versions are supported as well.
------------	--

Squish HTML	HTML file format reports with .html file extension are generated by Squish via command line which can be viewed in the web browser and can be uploaded to Squish Test Center. To generate the HTML report from Squish, run the test suite using squishrunner with the proper reportgen arguments: squishrunner --testsuite suite_addressbook_py --reportgen html,report_folder_name More information on generating test result via the squishrunner can be found in the course on Advanced Test Execution.
Coco CSMES/CSEXE	Code coverage report files generated from Coco like single instance of Coco instrumentation database *.csmes per report can be uploaded together with several corresponding execution reports *.csexe.

STEPS - Uploading Results from the Squish IDE

STEP 01 = Run tests in the Squish IDE



We started by running predefined tests from the Squish IDE to gather the tests results we wish to upload to Test Center.

STEP 02 = Start Test Center

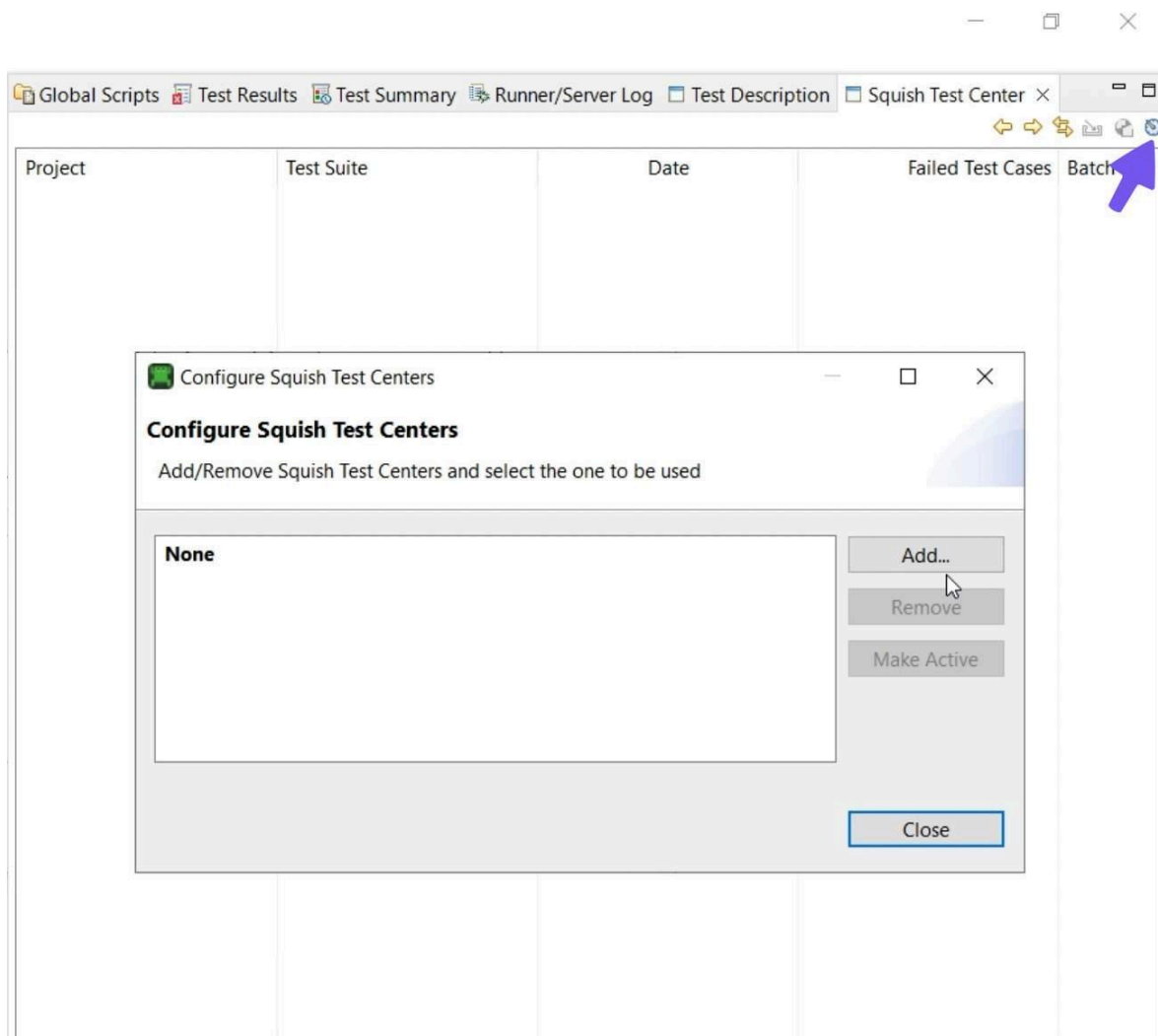
None

```
C:\squish-testcenter-3.3.0-windows-x64\bin>testcenter start
Reading database configuration for environment "product" from
"C:/squish-testcenter-3.3.0-windows-x64/testcenter/config/database.
ini"
DriverType: "SQLITE"
DatabaseName: "db/teamserver.dbfile"
Absolute SQLite file path "C:/squish-testcenter-3.3.0-windows-x64
/testcenter/db/teamserver.dbfile"
Connect Options: "SQLITE_BUSY_TIMEOUT=2147483647"
```

```
SQLite version: 3.24.0
Current Schema Version: "202306151745"
Latest Schema Version: "202306151745"
Test Center started, listening on port 8800
```

We started Test Center from the command line by issuing the command '**testcenter start**'. The Test Center server connection was confirmed as successful as we saw the output "Test Center started, listening on port 8800".

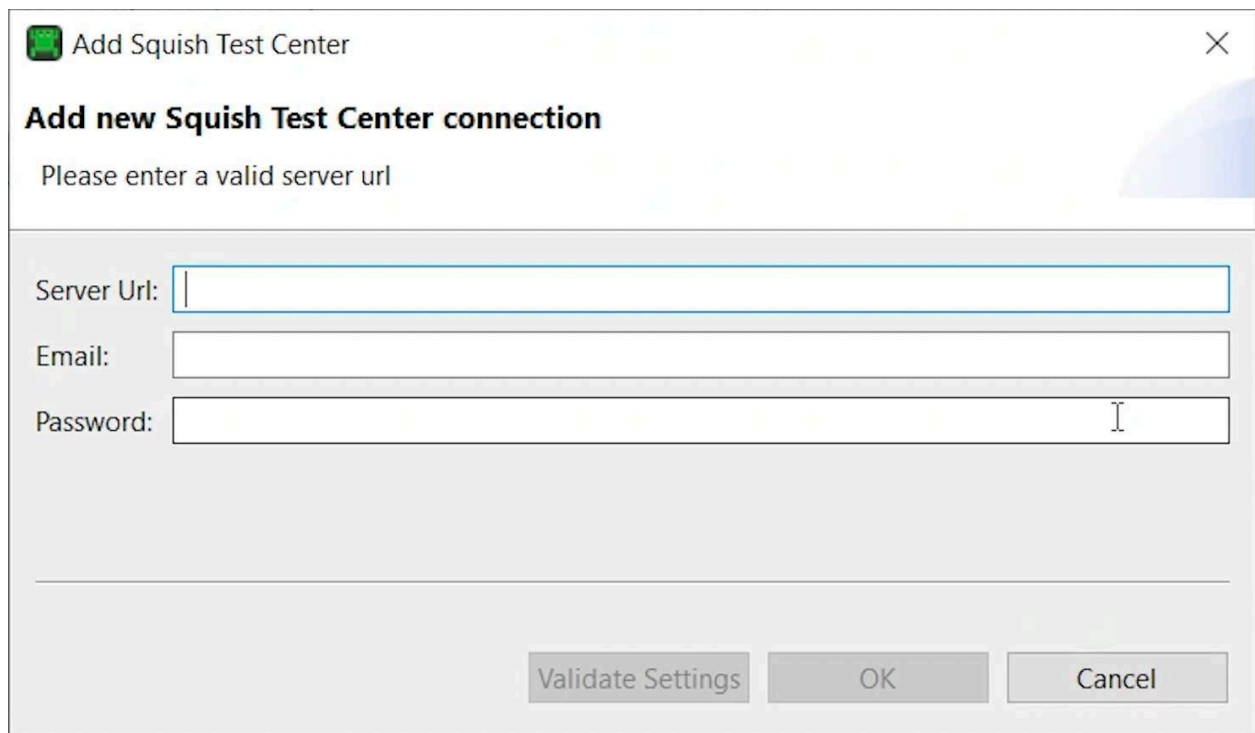
STEP 03 = Configure Test Center in the Squish IDE



From the Squish IDE, we configured our connection to the Test Center server. By default, the Squish Test Center tab is visible as seen below. If not, you can access it by navigating to **Window > Show View > Other > Squish Tests > Squish Test Center**.

We clicked on the '**Configure Squish Test Centers**' option to add our Test Center Server details.

STEP 04 = Validate and add settings

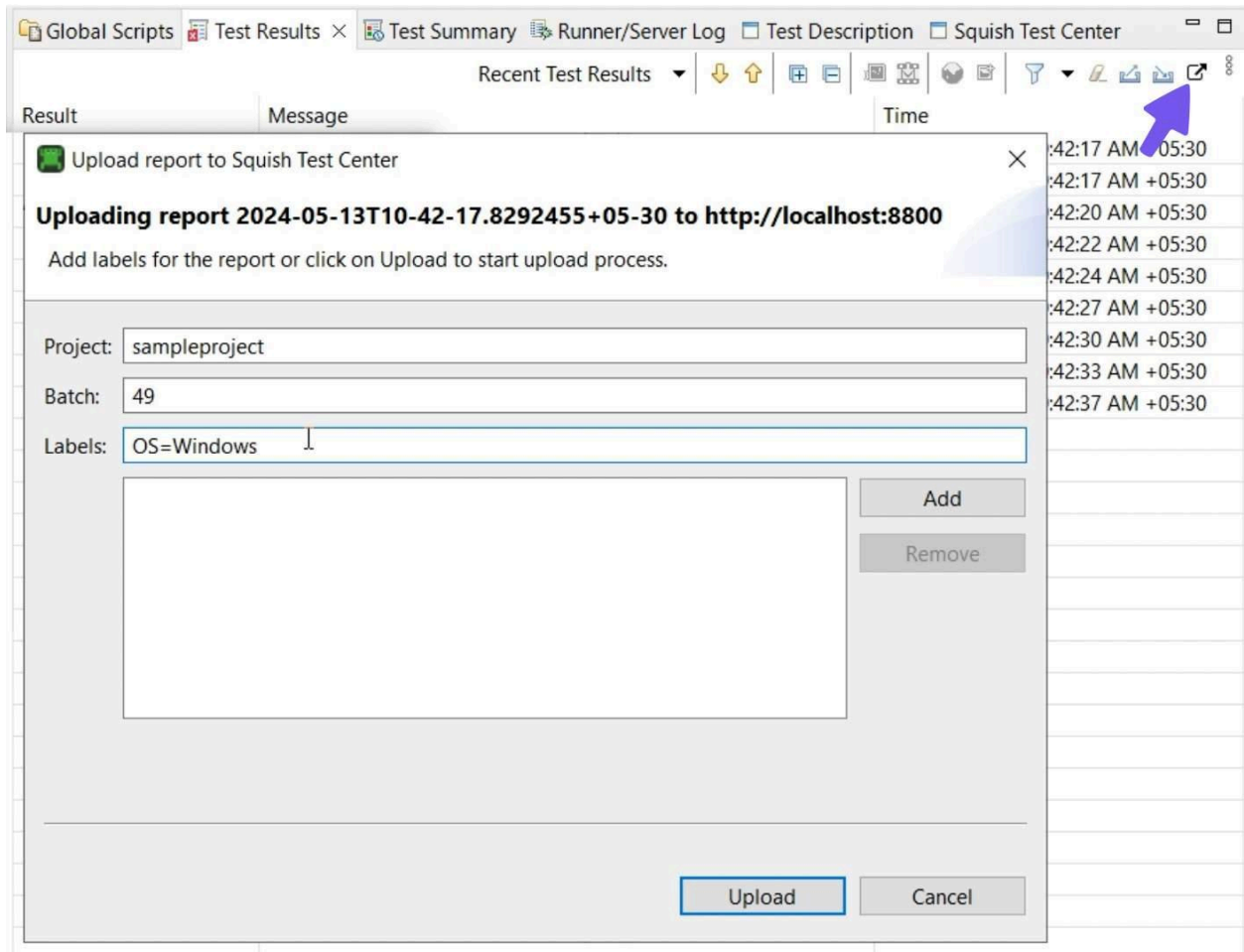


The screenshot shows a dialog box titled "Add Squish Test Center" with a close button (X) in the top right corner. Below the title bar, the text "Add new Squish Test Center connection" is displayed, followed by the instruction "Please enter a valid server url". The dialog contains three input fields: "Server Url:" with a text box, "Email:" with a text box, and "Password:" with a text box. At the bottom of the dialog, there are three buttons: "Validate Settings", "OK", and "Cancel".

After clicking the '**Add**' button, we entered the server URL including port number, along with our login credentials (email address and password).

Before clicking '**OK**', we validated the server settings to ensure our credentials were correct. A message appeared "**Successfully connected to the Squish Test Center using the specified credentials**".

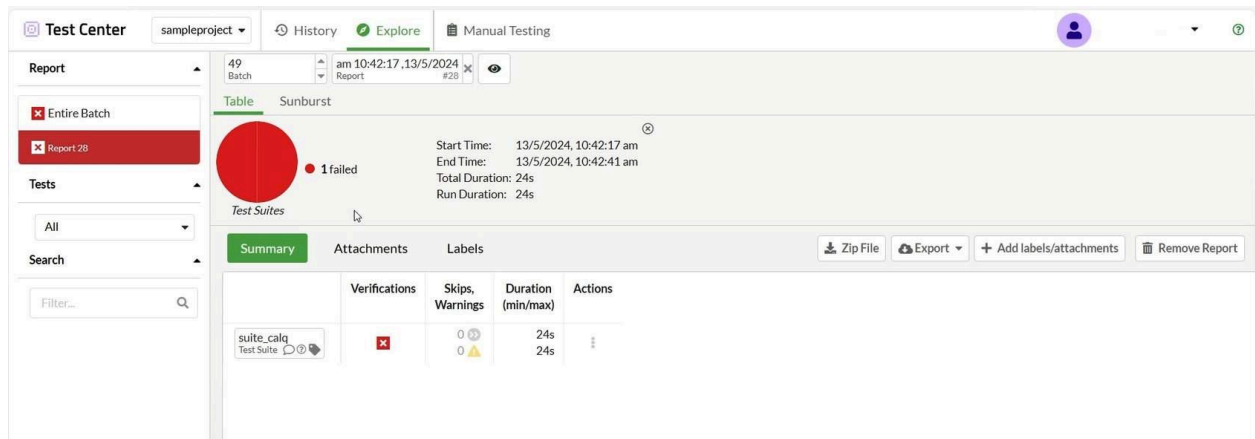
STEP 05 = Upload test results



In the Test Results tab, we selected the button '**Upload Result to Squish Test Center**'.

The '**Upload report to Squish Test Center**' dialog appeared, and we specified the **Batch** and any **Labels**, which help to group the test results. After uploading, we received a confirmation with a direct link to the results.

STEP 06 = View results in Test Center

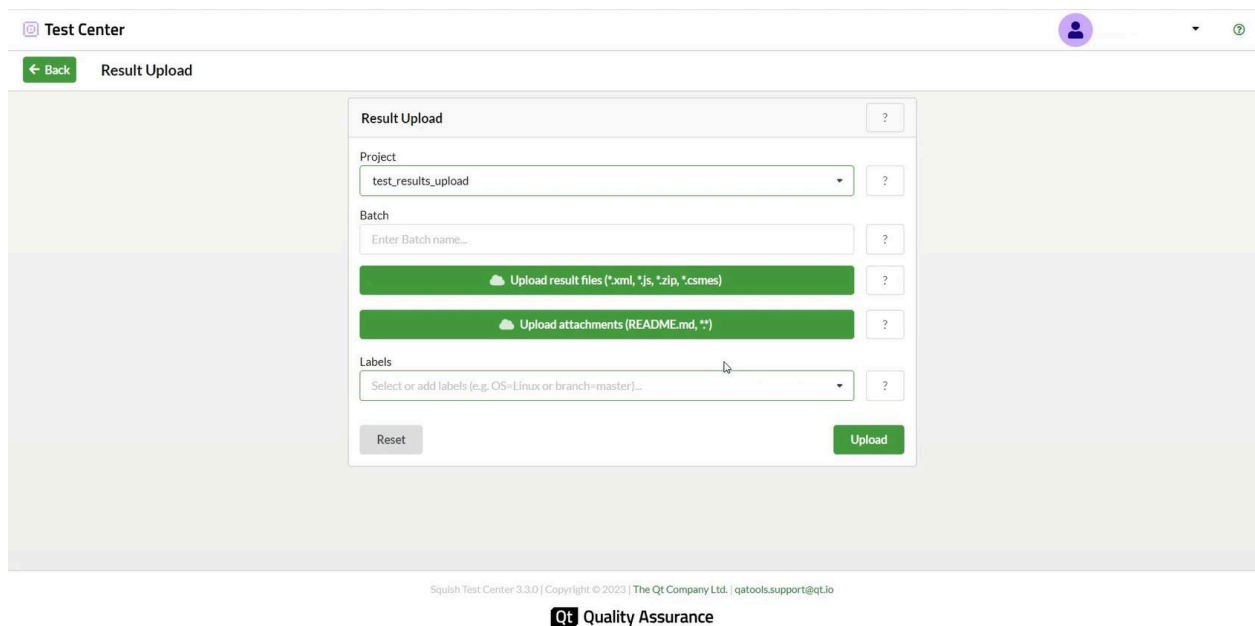


We then logged in to Squish Test Center to view the results.

Uploading Test Results Manually

In this section, we will focus on uploading test results manually to Squish Test Center.

Test results generated from the Squish IDE or another 3rd party IDE can be uploaded to Squish Test Center manually. For example, with the Squish IDE, test results can be exported as **zip** archive consisting of a **results.xml** file, If your result contains screenshots, snapshots or attachments, we upload the whole **.zip** file.



The screenshot shows the 'Test Center' web interface with a 'Result Upload' modal window. The modal contains the following fields and buttons:

- Project:** A dropdown menu with 'test_results_upload' selected.
- Batch:** A text input field with the placeholder 'Enter Batch name...'.
- Upload result files (*xml, *js, *zip, *csmes):** A green button with a cloud icon.
- Upload attachments (README.md, *):** A green button with a cloud icon.
- Labels:** A dropdown menu with the placeholder 'Select or add labels (e.g. OS=Linux or branch=master)'.
- Reset:** A grey button at the bottom left.
- Upload:** A green button at the bottom right.

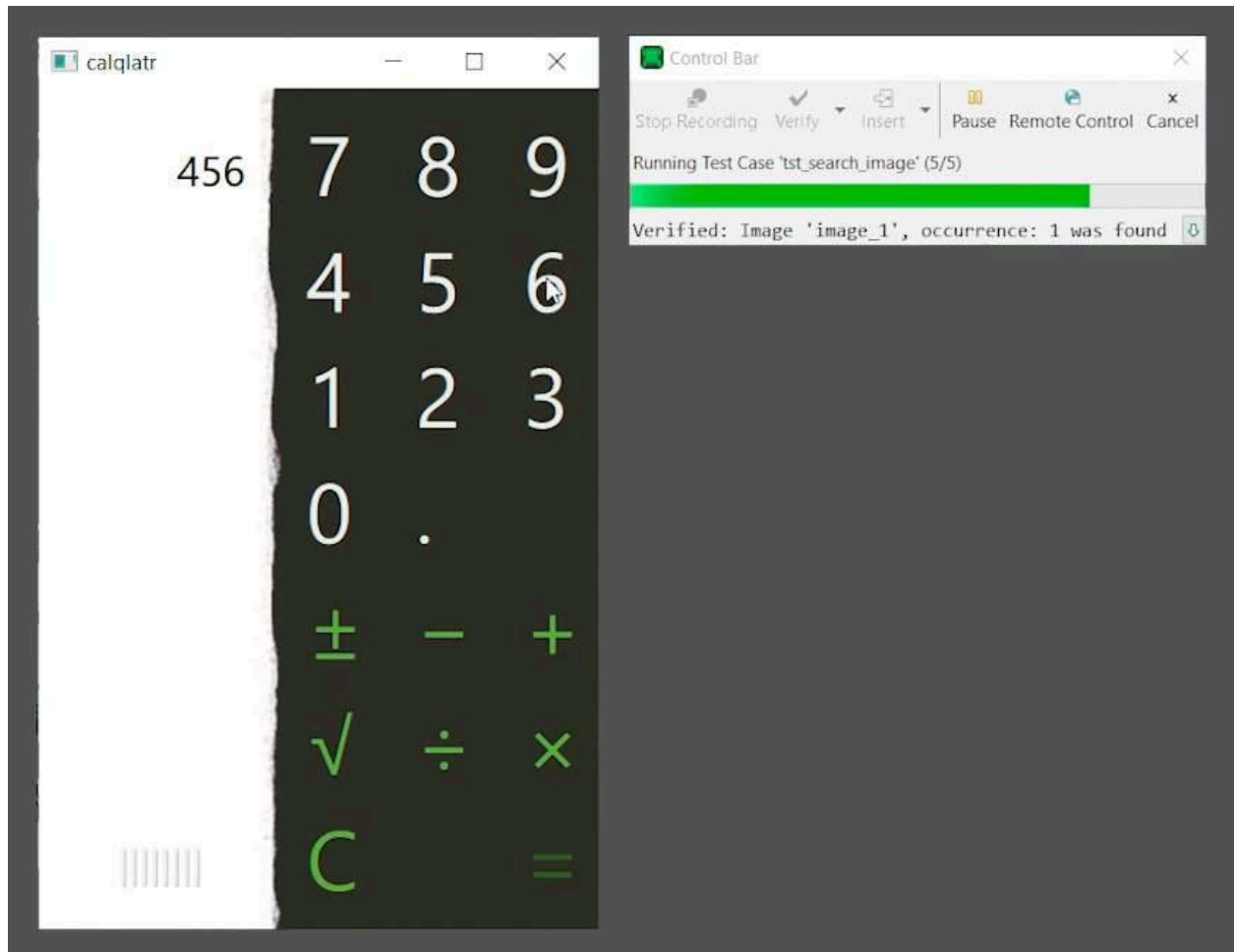
At the bottom of the page, there is a footer with the text: 'Squish Test Center 3.3.0 | Copyright © 2023 | The Qt Company Ltd. | qatools.support@qt.io' and the 'Qt Quality Assurance' logo.

Test Results from the Squish IDE can be exported by using the export option from the Test Results tab in Squish IDE. Apart from XML file format Squish Test Center supports the other formats as discussed in previous section.

Test results can be uploaded to Squish Test Center manually by going to the drop-down menu next to the user icon and selecting **'Result Upload'**. From here the results file and attachments can be uploaded and the project name, batches, and labels can be defined.

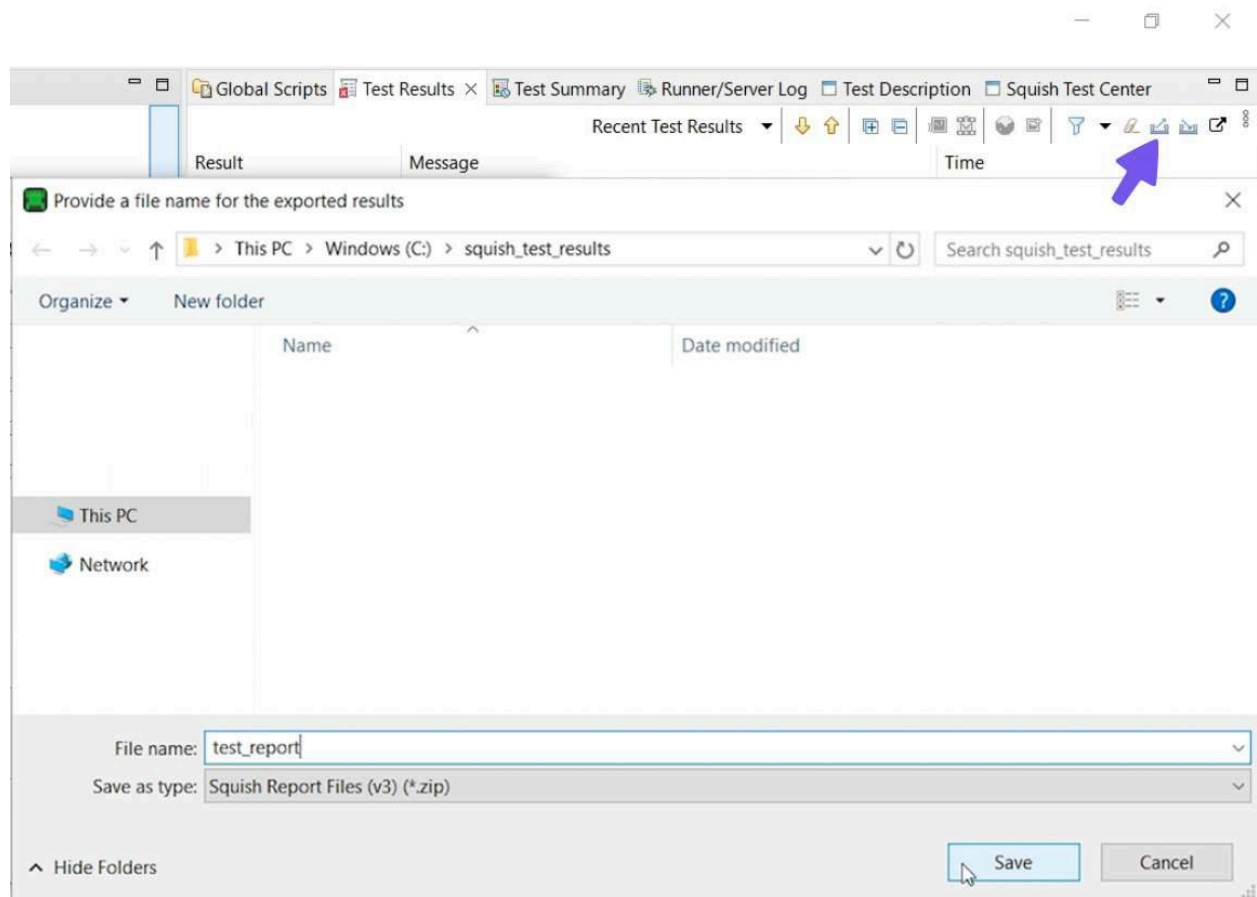
STEPS - Uploading Results from the Squish IDE

STEP 01 = Execute tests from the Squish IDE



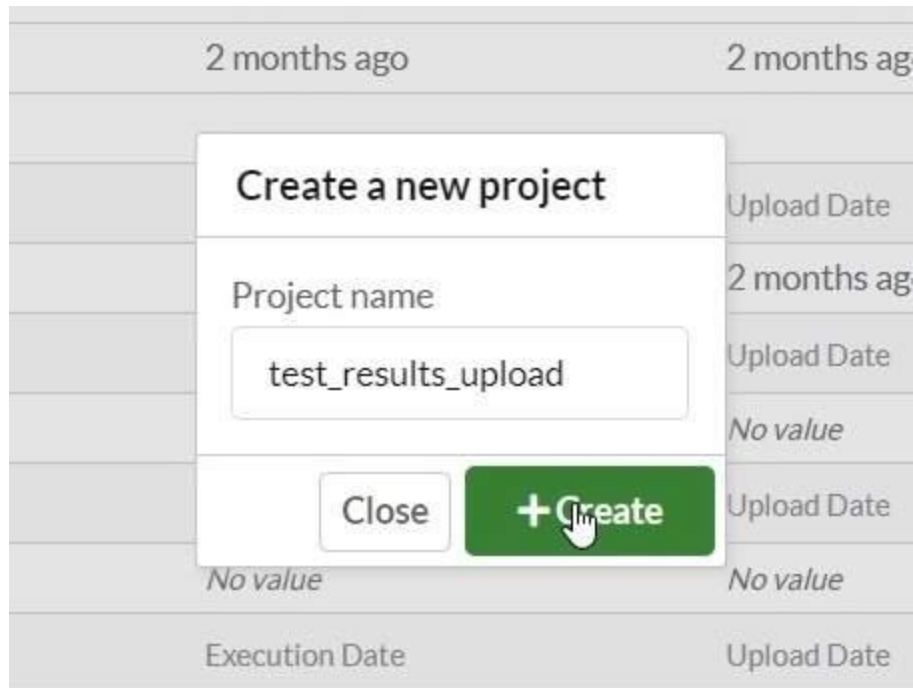
We started by running predefined tests from the Squish IDE to gather the tests results we wish to upload to Test Center.

STEP 02 = Export test results



In the Test Results tab, we selected the button '**Export**'. The file explorer dialog appeared to save the **zip** archive of our test results to our desired location.

STEP 03 = Create a new project in Test Center



With Test Center running from the command line, and open in a web browser, we clicked on the **'Add Project'** button in the project dropdown. We entered the name as **“test_results_upload”** and once the confirmation dialog appeared, we selected the **'Upload results'** button .

STEP 04 = Upload test results

Result Upload

?

Project

test_results_upload

?

Batch

B24

?

Upload result files (*.xml, *.js, *.zip, *.csmes)

test_report.zip

Upload attachments (README.md, *)

?

Labels

OS=Windows x

?

Reset

Upload

We now entered a **batch** name and **labels**, and attached the test results **zip** archive, uploading the test results to our newly created project.

STEP 05 - View test results

Test Center

test_results_upload

History

Explore

Manual Testing

Report

Entire Batch

Windows

Tests

All

Search

Filter...

OS

Windows (1)

B24

AM 11:54:35, 5/13/2024

Report

#29

Table

Sunburst

1 failed

Start Time: 5/13/2024, 11:54:35 AM

End Time: 5/13/2024, 11:54:58 AM

Total Duration: 23s

Run Duration: 23s

Summary

Attachments

Labels

Zip File

Export

Add labels/attachments

Remove Report

	Verifications	Skips, Warnings	Duration (min/max)	Actions
suite_calq Test Suite		0 0	23s 23s	

We can now view the results. If more test results are exported, we now know they can also be added to any existing project in Test Center by using the corresponding project name.

Uploading Test Results Via Command Line

In this section, we will focus on uploading test results to Squish Test Center via command line.

The advantage of uploading via command line is it allows the user to upload the results from a batch script, also it can be used for automated testing like regression testing. You can upload results to Squish Test Center on the command line by using `testcentercmd` which is a command line tool available in the Squish Test Center installation folder which allows a user to quickly upload the test results without using the UI. A Test Center server instance can be invoked from the command prompt to upload the test results. There is a standard set of arguments which must be followed while using the command prompt, the following which are required:

- An upload token or the email address and password of a Squish Test Center user with upload privileges.
- Project name that identifies tests of the same software, project, or module.
- Path to the test result file or the result directory.

The following are optional:

- URL of the server instance, the default being `http://localhost:8800`.
- One or more labels to filter the test results of the same project, the default being none.
- Name of a batch instead of a generated name

For Example:

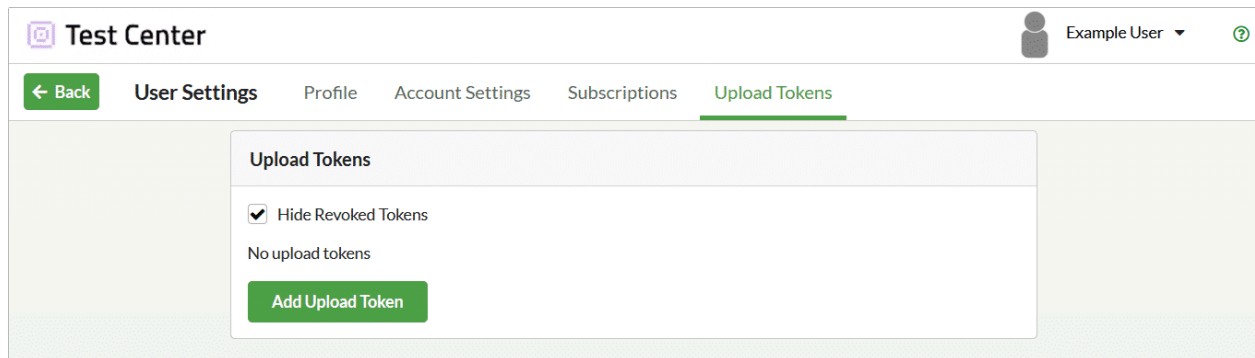
None

```
testcentercmd --url=http://localhost:8800 --user=mailbox@email.com  
--password=myPassword upload Myproject /path/to/results.zip  
--label=version=1.0.0  
--label=OS=Windows --batch=MyBatch
```

Create and Manage Upload Tokens

To create [upload tokens](#), select **Upload Tokens** in the **User Settings** view, and then click **Add Upload Token**. This option and tab are available only for users with the upload role set in the User Management view.

A name can be assigned to each of the generated tokens to identify who is using which tokens. Tokens can be copied and edited according to requirements. Below shows the Upload Tokens section in Test Center.



STEPS - Uploading Results Via Command Line

STEP 01 = Uploading results using testcentercmd

Shell

```
testcentercmd --url=http://localhost:8800 --user=mailbox@email.com  
--password=myPassword upload commandprj /path/to/results.zip --label=version=3.0.0  
--label=OS=Windows --batch=Batch24
```

We uploaded our test results zip archive using from the command line using the **testcentercmd** executable which is shipped with Squish Test Center. We passed the url of the server, username and password, using '**commandprj**' as the project name, along with labels to specify version and our operating system, and a batch number.

After we passed the command with appropriate arguments we received the overall status of the tests defined and imported.

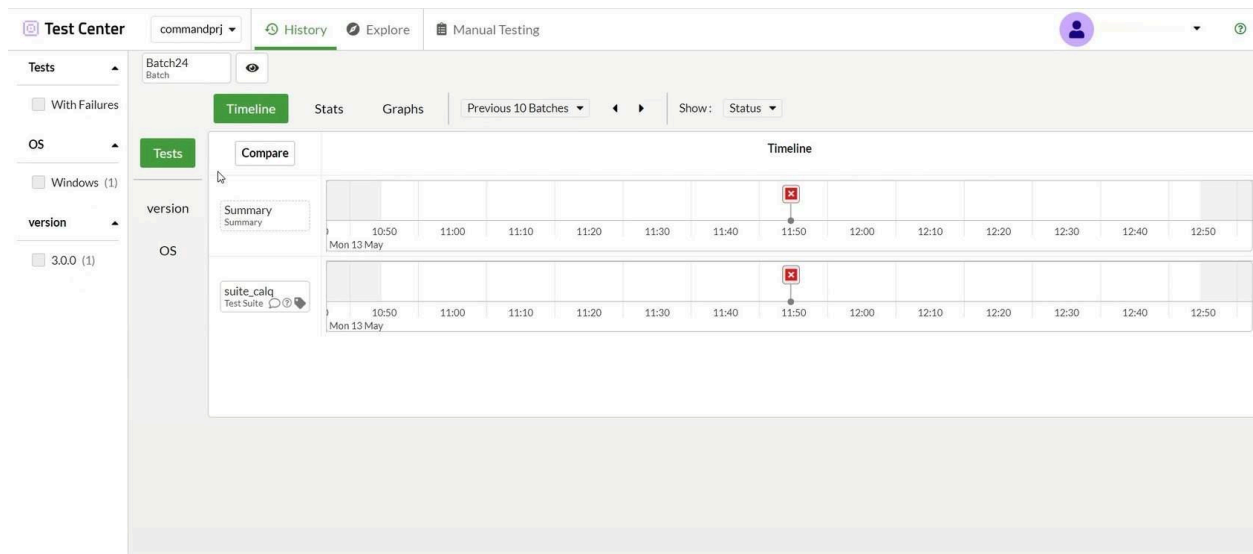
STEP 02 = Launch Test Center

Shell

```
C:\squish-testcenter-3.3.0-windows-x64\bin>testcenter start
Reading database configuration for environment "product" from
"C:/squish-testcenter-3.3.0-windows-x64/testcenter/config/database.ini"
DriverType: "SQLITE"
DatabaseName: "db/teamserver.dbfile"
Absolute SQLite file path "C:/squish-testcenter-3.3.0-windows-x64
/testcenter/db/teamserver.dbfile"
Connect Options: "SQLITE_BUSY_TIMEOUT=2147483647"
SQLite version: 3.24.0
Current Schema Version: "202306151745"
Latest Schema Version: "202306151745"
Test Center started, listening on port 8800
```

We started Test Center from the command line. The Test Center server connection was confirmed as successful as we saw the output "Test Center started, listening on port 8800".

STEP 03 = View uploaded results



After navigating to Squish Test Center via our web browser and entering our credentials, we can see the project "commandprj" created successfully with batch name as "Batch24" and "OS=Windows" as a label.

When logging into Test Center, you are greeted with the program's dashboard, which is a fully customizable view of the server. Let's explore some of the views in more detail:

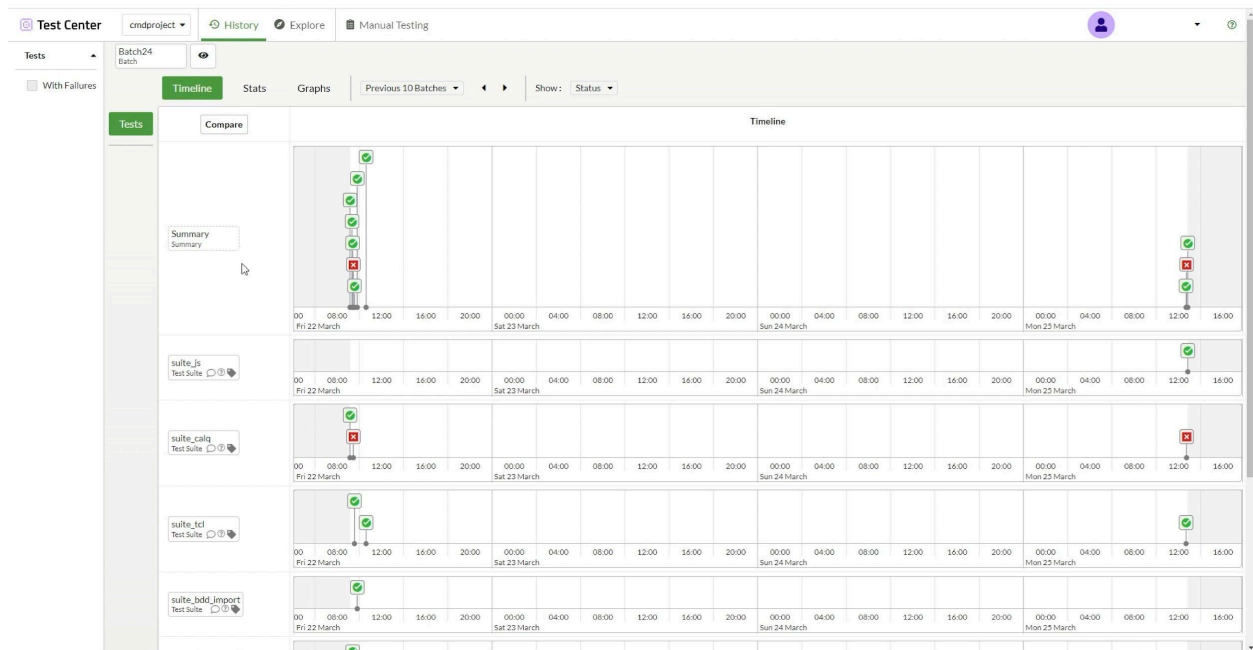
The [Dashboard](#) view shows a high-level view of the projects, as well as the most recent Activity Stream that displays login details and results that have been uploaded. In the drop-down menu in the top-left, a new project can be created, or an existing project can be opened. When clicking on one of these projects, the user will be taken to the History view, which is made up of the timeline, statistics, and graph views.

History

The [History](#) view allows the user to go back to a particular point in time and view the results or series of results for a particular range, beginning at that point in time. The breadcrumbs bar from a particular test suite is available in the History view with the convenience of switching between test suites and test cases of a particular test suite.

Timeline

With the **Timeline**, you can configure the time frame shown for each section. You can choose one of the following preconfigured time frames from **Today** to the **Last 10 Batches**. Or, select **Custom Range** and pick a **Start Date** and an **End Date**. In the timeline, there is a status icon which represents each batch for the time frame selected. The status icon indicates if the batch **passed** ✓, **failed** ✗, was marked as an **unexpected pass** (indicated by a red comment bubble with an '✗'), or was marked as an **expected failure** (indicated by a green comment bubble with a tick '✓'). Clicking the icons opens the Explore view, to learn more about the execution of the test.



Stats

Stats provides the statistics for batches of a particular test suite or a particular test case. Stats presents various statistics on a test. All metrics ignore retried and skipped tests. There are several metrics to explore different aspects of a test:

Flakiness	Measure the stability of the test in terms of changes in state (pass --> fail), number of executions (fail --> pass), and the flakiness (m/n) of the test. For example, if there are N iterations and M iterations, and the result changes in both directions, then the flakiness will be M/N.
Success Rate	Given the number of passed runs P and total number of runs N, the success rate is P/N.
Time Broken	Indicates the number of times a failure occurred, not necessarily the number of failures incurred.
Retry Count	Total number of times a test was retried.
Skip Rate	Given number of skipped runs S and total number of runs N, the skip rate is S/N.
Average Duration	Lists the suite's runtime, averaged against all test executions within the specified time period.
Average time to fix after failure	Shows the average time until the tests recovered to a passing state.
Average runs between fixes	Shows the average number of runs until the tests recover to a passing state.

Batch24
Batch

Timeline

Stats

Graphs

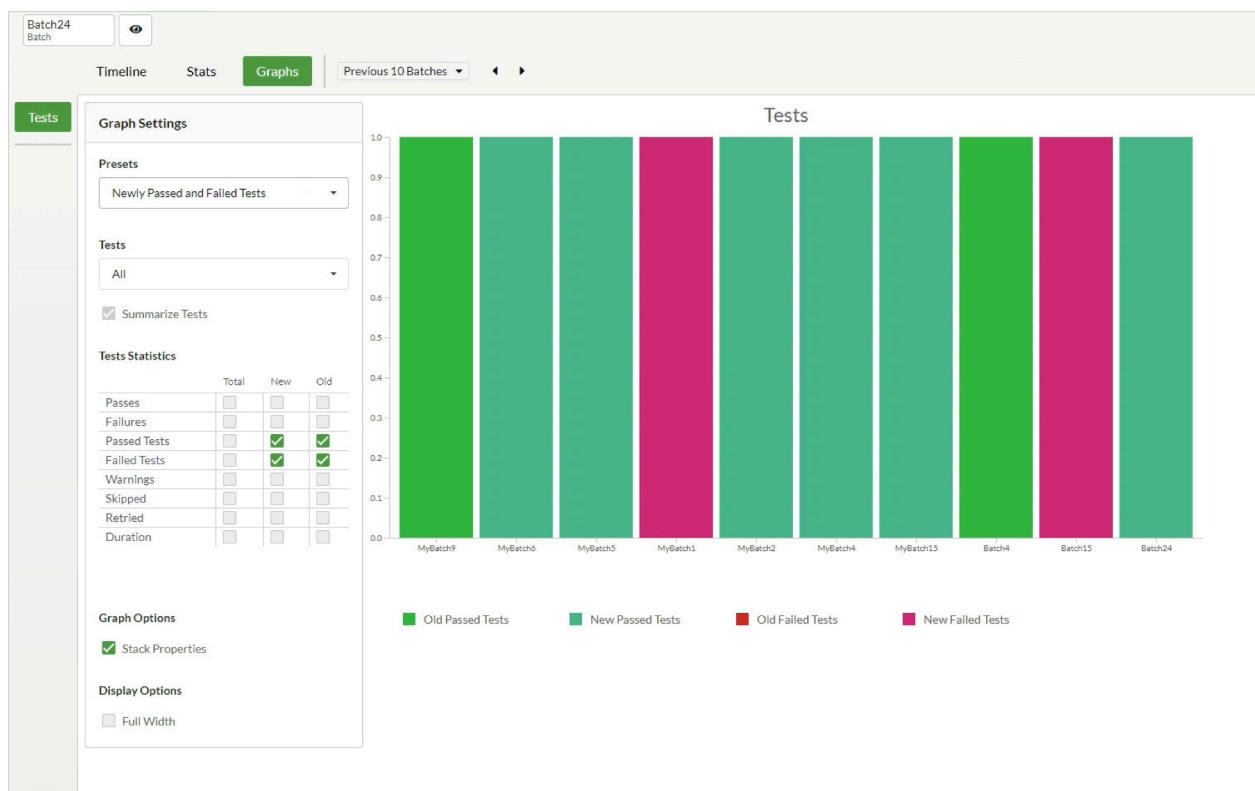
Previous 10 Batches

	Flakiness	Success Rate	Times broken	Retry Count	Skip rate	Avg duration	Avg time to fix after failure	Avg runs between fixes
suite_js Test Suite	0.00	100.00%	0	0	0.00	24s	-	-
suite_calq Test Suite	0.33	33.33%	1	0	0.00	25s	-	-
suite_tcl Test Suite	0.00	100.00%	0	0	0.00	22s	-	-
suite_bdd_import Test Suite	0.00	100.00%	0	0	0.00	17s	-	-
suite_quickaddressbook.js Test Suite	0.00	100.00%	0	0	0.00	6.1s	-	-
suite_keyword_py Test Suite	0.00	100.00%	0	0	0.00	8.6s	-	-

Graphs

In the **Graphs** view, the user will find graphs for many of the test properties. By using **Presets**, the user can choose a pre-defined graph, like **Total passes/failures** or **Test duration by tests**. To set up a **custom** graph, click **Custom** and then go to **Test Statistics** and select the test properties. The test properties that can be chosen from include **Passes/Fails**, **Passed/Failed**, **Warnings/Skips/Retries**, and **Duration**.

To sum up all the properties of all the tests or labels selected, click **Summarize Tests** or **Summarize Labels**. If this option is disabled, each test or label will be listed individually in the chart.



Explore

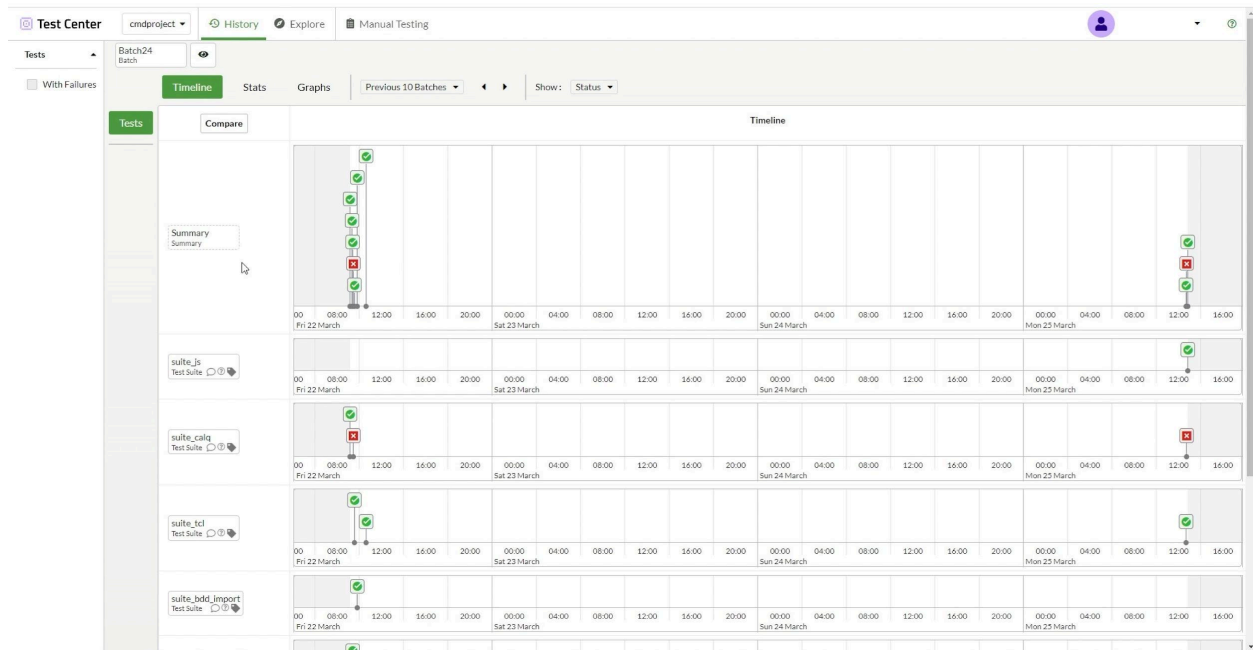
The Explore view contains a detailed overview of your projects' tests, organized by **Batch** and filtered by the **Label** you set during result upload. You can view the uploaded results in a number of ways.

In the **Breadcrumbs bar** (located just under the explore tab) , users can see the granularity of the view and identify the tests being looked at. In the first element of Breadcrumbs Bar, the name of the Batch is displayed, which can be switched by adjusting this field.

In the reports and tests table, users can see whether a test **passed** , **failed** , was marked as an **unexpected pass** (indicated by a red comment bubble with an ''), or was marked as an **expected failure** (indicated by a green comment bubble with a tick '') across all the uploaded reports of a batch. The number of total skips, the number of warnings, and the warning changed since the last execution are shown in the table. Additional functionality can be accessed with **Actions**. Issues/test items can be created in the third-party system if Jira/Zephyr integration has been activated.

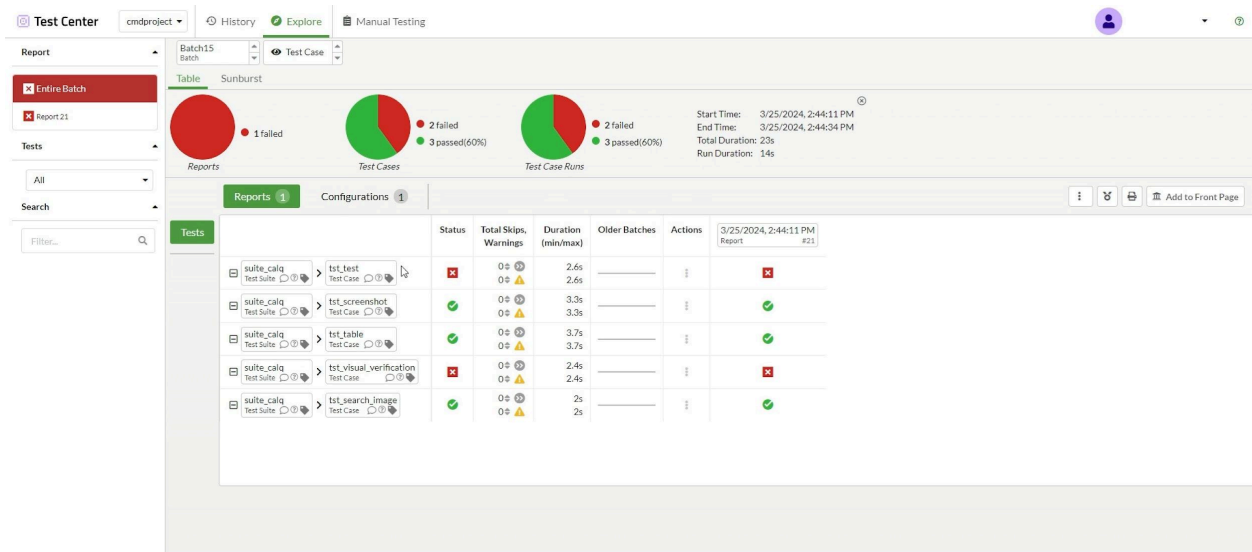
menu, you could create a new project or open an existing one. Clicking on a project took the user to the History view, which included timeline, statistics, and graph views.

STEP 02 = Examining the History view



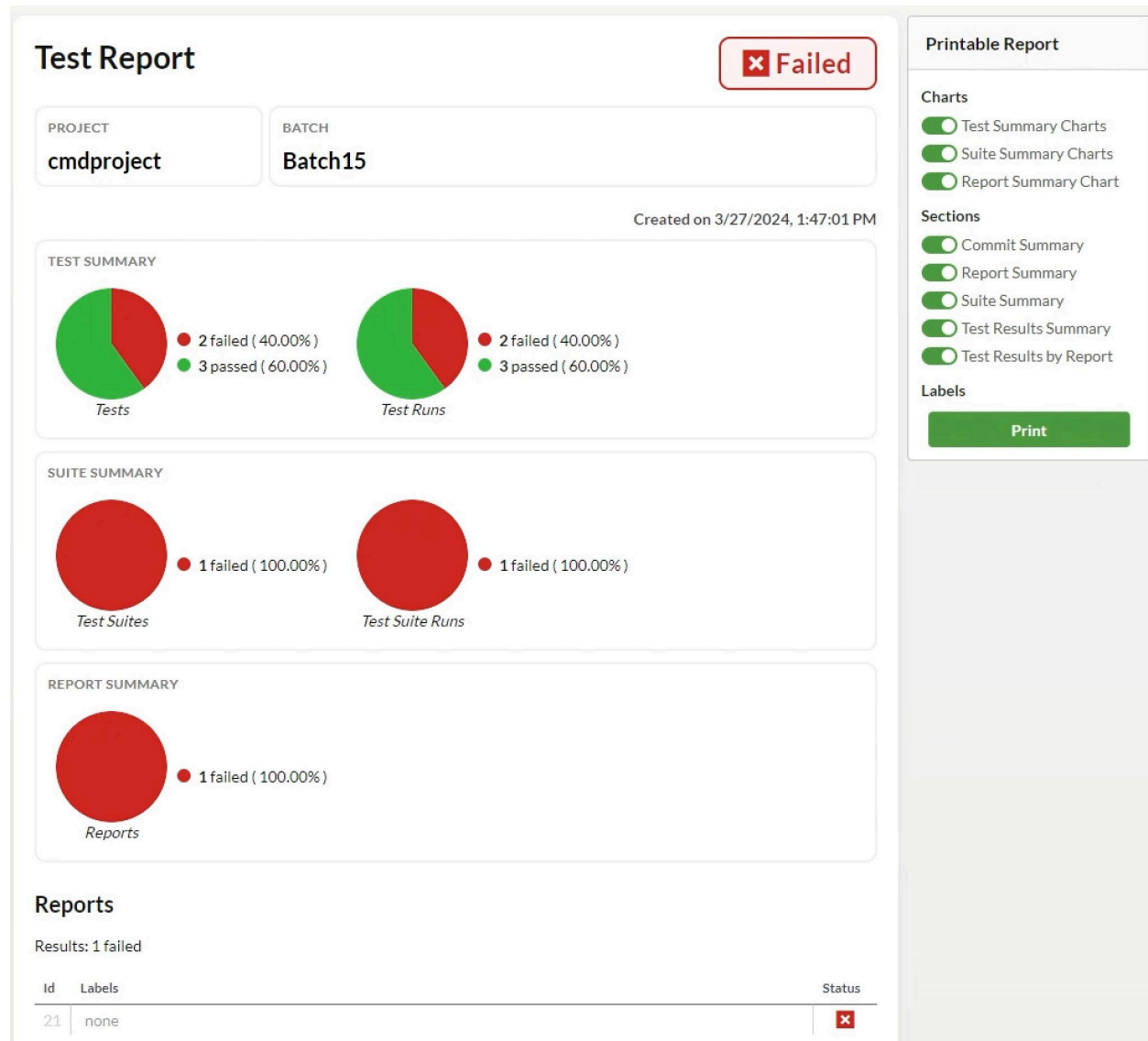
The History view allowed you to go back in time and view results for a specified range, with the ability to switch between test suites and cases. The Timeline enabled configuration of time frames, displaying status icons for each batch, indicating pass, fail, unexpected pass, or expected failure, and linking to detailed test execution in the Explore view. Stats provided various metrics for test suites or cases, excluding retried and skipped tests, allowing exploration of different test aspects.

STEP 03 = Examining the Explore view



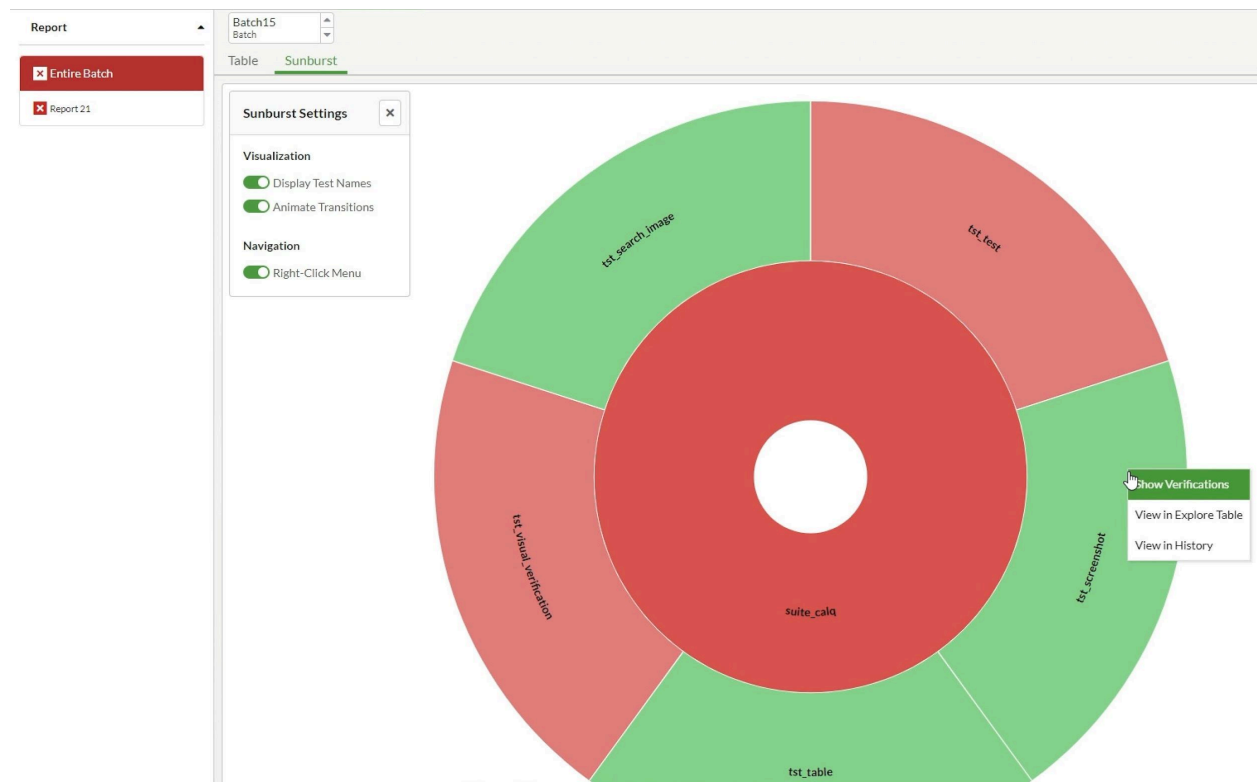
The Explore view provided a detailed overview of project tests, organized by Batch and filtered by Label set during result upload. You could view uploaded results in various ways. The Breadcrumbs bar displayed the granularity of the view and the specific tests, with the Batch name shown in the first element, which could be adjusted. The reports and tests table indicated whether a test passed, failed, was an unexpected pass, or an expected failure, along with the total skips, warnings, and changes since the last execution. Additional functionality included creating issues/test items if Jira/Zephyr integration was activated.

STEP 04 = Creating a printable report



Clicking on the print button in the explore view, we could generate a test report in PDF format, consisting of a test summary with passed and failed tests. Options to select charts and sections could be added to the report. The report also included the test suite name and test case status, indicating pass or fail.

STEP 05 = Examining the Sunburst view



The sunburst view displayed a pie chart with green indicating pass and red indicating fail. Each slice represented a test item, such as the “suite_calq” we explored. The test suite was in the center surrounded by test cases. The view included settings to enable or disable test names and transition animations for navigating test items. Right-clicking on any test case allowed us to jump to verifications, history, and table views.

-Manual Testing in Test Center

This section will examine how to perform manual testing in Test Center. In the [Manual Testing](#) view, you can manage the test step descriptions of manual tests and execute the tests to analyze the results in the other views.

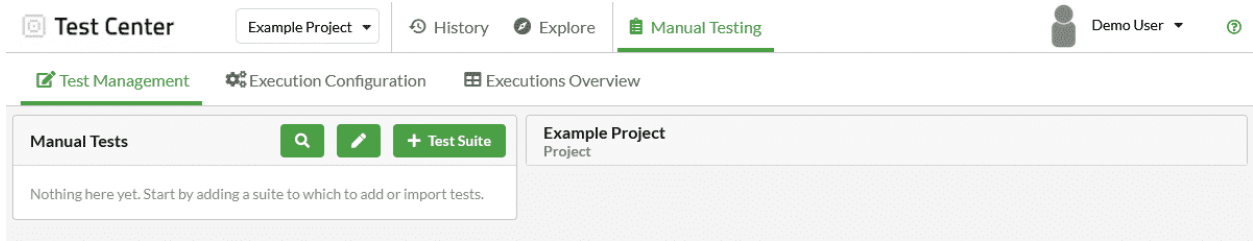
First, you need to select or create a project. If you already manage automated test results in Squish Test Center, you can use the same project you use for your automated test results. Otherwise, create a new project.

To open the view, select Manual Testing.

Test Management

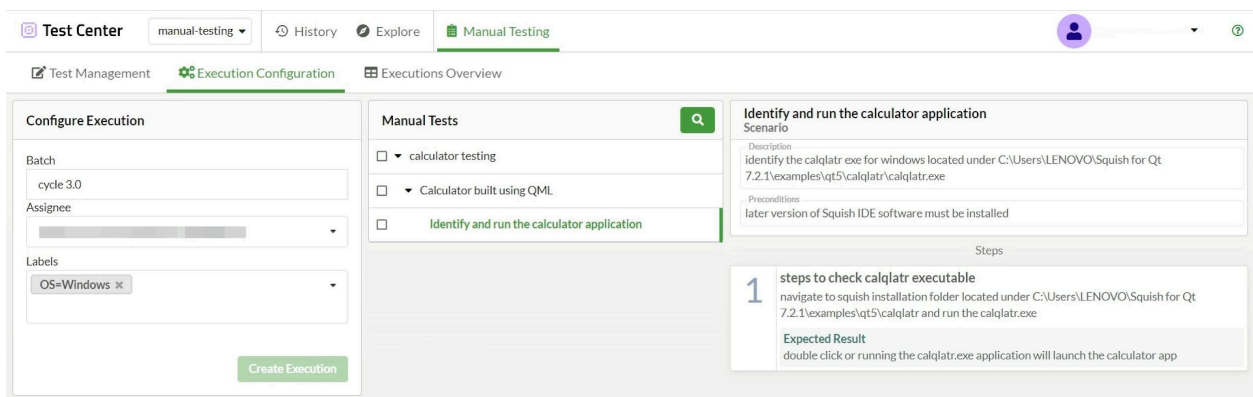
The **Test Management** view allows you to manage the test suite configurations by adding, removing, or editing test suites and test steps. In the Manual Tests pane on the left, new test suites can be created with or without a BDD (Behavior-Driven Testing) test structure.

Once the test suite is created successfully, you can edit the scenario with relevant steps by adding a title, description, and expected results to the Test Management tab. Once the manual test steps are added to the Test Management tab, the next step is configuring the execution.



Execution Configuration

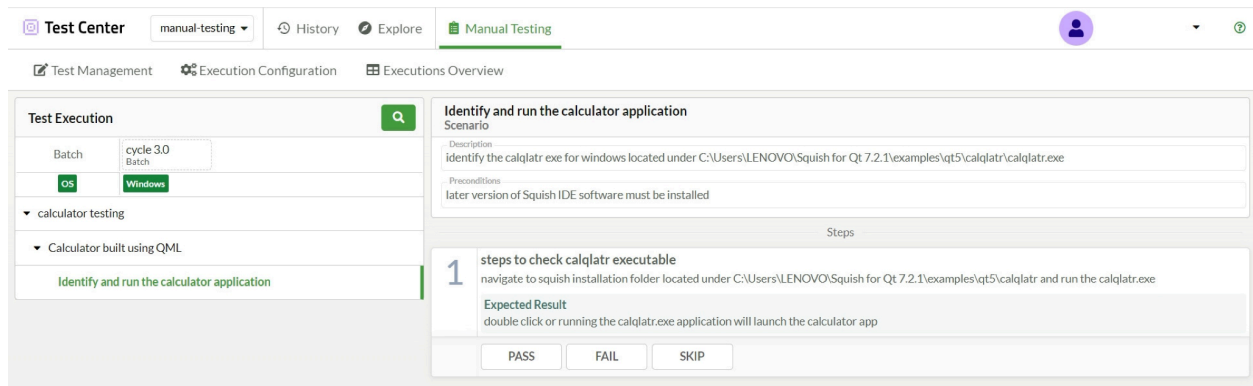
The **Execution Configuration** view allows you to configure the execution by selecting the manual test steps with additional options such as the Batch name, Assignee and Labels. In the Manual Tests section, select the test suites, test cases, or scenarios you want to include in the test cycle you are preparing. The right-hand side test details pane displays more information about the selected test and its steps.



Test Execution

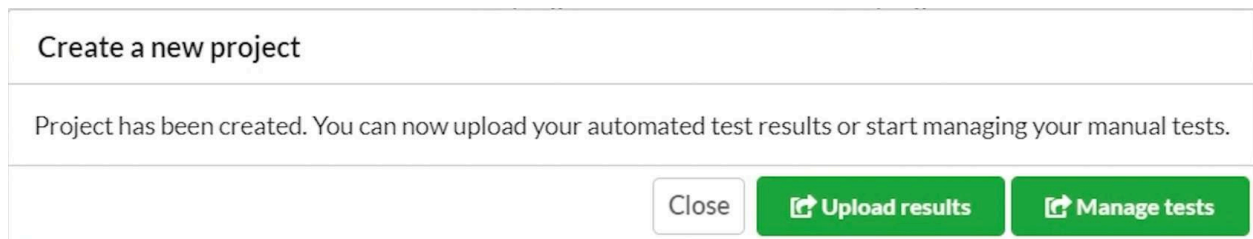
After clicking on play button, the Test Execution view lets you work through the individual steps of a manual test execution. You can go through the steps manually, if you are able to perform the steps as described and if the performed steps are successful, click on **PASS** button, otherwise click on **FAIL**, if you wish to skip this Test Execution step, click on **SKIP**. After choosing the outcome, click on **Finish Execution**.

After execution, you can go to the report which leads to Explore view, which helps in analyzing the executed Manual Testcase.



STEPS - Manual Testing in Test Center

STEP 01 = Creating a new project



After creating a new project from the project dropdown menu, we were greeted with a confirmation dialog. For manual testing, we chose the option **Manage tests**. The **Manual Testing** tab opened with our newly created project.

STEP 02 = Create a new test suite

New Suite

Suite Name

calculator testing

Use BDD Test Structure (Behaviour Driven Testing)

Create Suite

In the **Test Management** tab for our project, we created a new test suite to begin constructing our manual testing.

STEP 03 = Specifying the feature

Test Centermanual-testingHistoryExploreManual Testing

Test ManagementExecution ConfigurationExecutions Overview

Manual Tests

calculator testing

Calculator built using QML

Calculator built using QML

Feature

Description

verify the integrity of calculator application

Preconditions

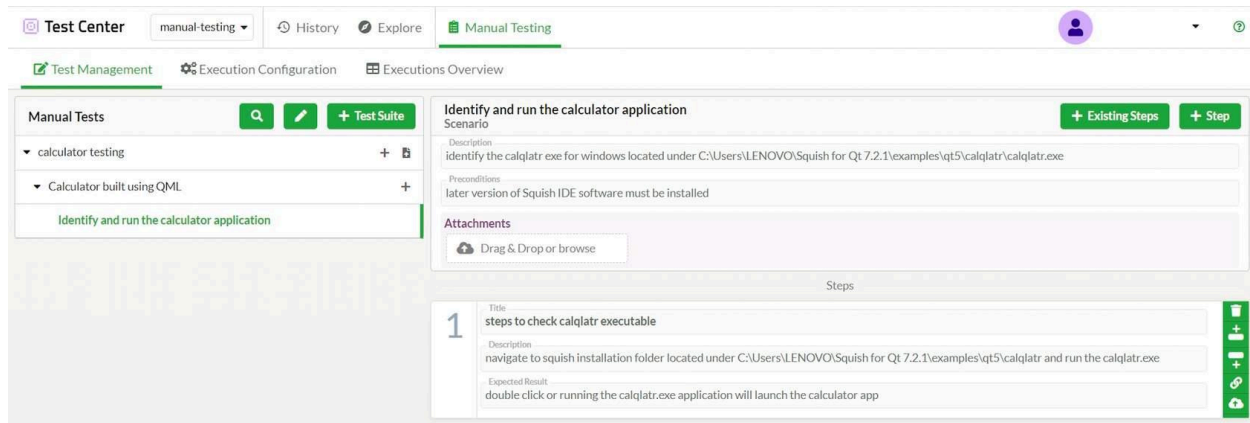
application works only on windows OS environment

Attachments

Drag & Drop or browse

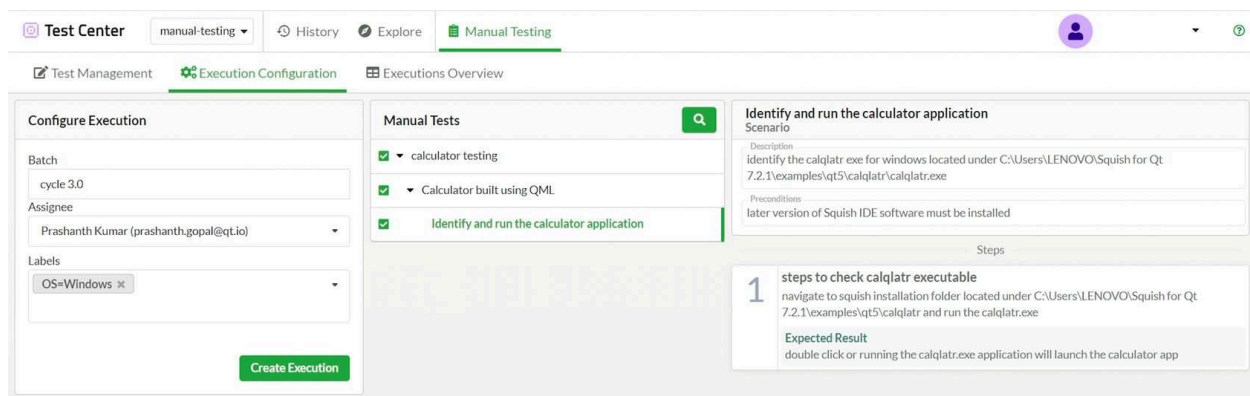
After creating our test suite, we specified the the feature for the test case. In our example. we wanted to verify the integrity of the calculator application built using QML.

STEP 04 = Specifying scenario and steps



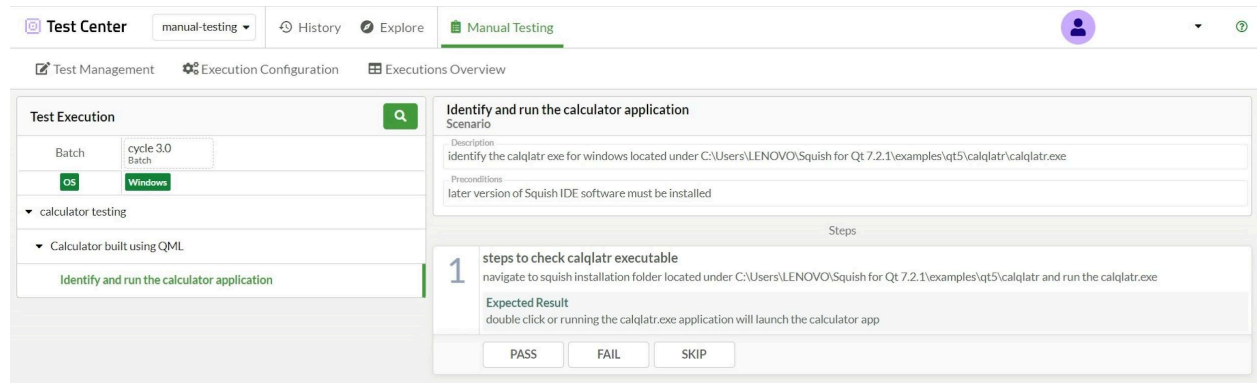
We then specified the scenario for the test conditions and the steps needed to be undertaken in order to verify our test. In our example, we were locating the calculator example that can be found in our up-to-date Squish installation and the expected outcome when launched.

STEP 05 = Configuring execution



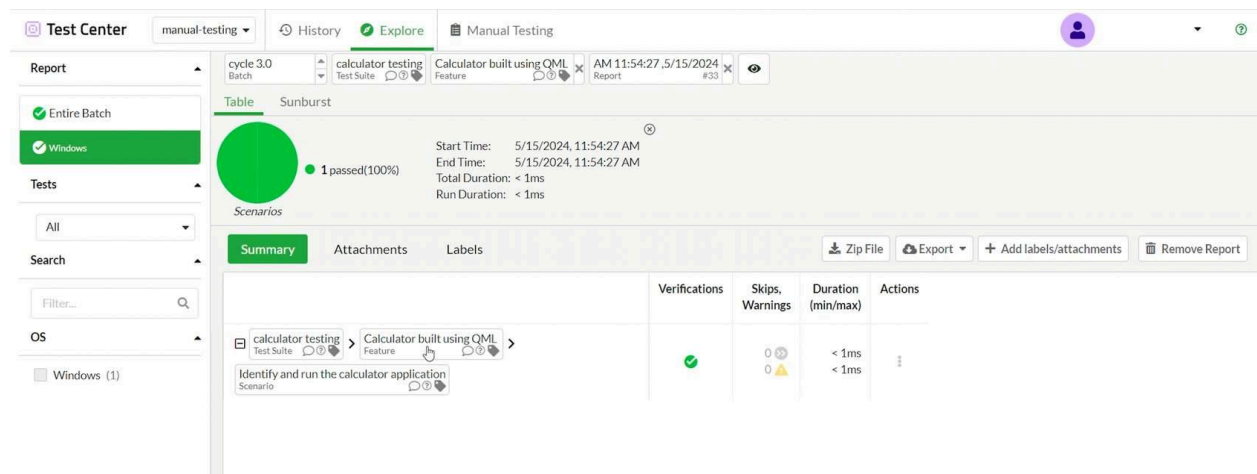
We then specified a batch, labels, and who the test execution is assigned to in the Execution Configuration view, selecting the test case, feature, and scenario assigned to the test execution.

STEP 06 = Executing the test



After executing the tests from Execution Overview, we familiarised ourselves with the test scenario and steps. We went through the steps manually to perform the steps as described, and if they were successful, we clicked on the PASS button; otherwise, we could click on FAIL. If we wished to skip this Test Execution step, you clicked on SKIP. After choosing the outcome, we clicked on Finish Execution.

STEP 07 = View the results



We were then able to go to the report to review the test results.

Summary

Getting Started with Test Center

Throughout this course, you've learned the core steps to get started with Test Center. You have configured the essential settings, such as the SMTP server, by editing the application.ini file, ensuring that your Test Center can communicate effectively. You have learned how to launch Test Center via the Command Line, activate the server, and set up user accounts.

Upload Options

A crucial part of your journey involved understanding the multiple methods available for uploading test results to Test Center. You explored three primary upload options: using the Squish IDE, manually uploading via the Test Center UI, and leveraging the command line tool testcentercmd. Each method serves different needs; for instance, the IDE option allows seamless integration during test execution, the manual method offers flexibility for third-party test results, and the command line tool supports automation and scripting for continuous integration workflows. These varied approaches provide a comprehensive toolkit to ensure that your test results are uploaded efficiently and correctly, regardless of the testing environment or requirements.

Navigating Test Center

Understanding Test Center's views is essential for leveraging its full potential. You have explored the dashboard to get an overview of projects and recent activities, explore detailed test results in the History view, and utilized the Stats tab to analyze metrics such as flakiness, success rate, and average test duration. The Graphs tab has shown you how to visualize test properties and generate custom reports, while the Explore view has provided insights into batch details and test case statuses. Additionally, you gained hands-on experience with the Manual Testing feature, enabling you to create, manage, and execute manual test cases. This ensures that you can efficiently manage and analyze your testing efforts within Test Center, leading to more informed decision-making and improved test management.